```
const express = require('express');
const session = require('express-session');
const helmet = require('helmet');
const bcrypt = require('bcryptjs');
const Database = require('better-sqlite3');
const multer = require('multer');
const path = require('path');
const fs = require('fs');

const app = express();
const PORT = Number(process.env.PORT || 3000);
const DB_PATH = process.env.DB_PATH || path.join(__dirname,
'universal-estate.sqlite');
const UPLOAD_DIR = path.join(__dirname, 'uploads');
fs.mkdirSync(UPLOAD_DIR, { recursive: true });
const db = new Database(DB_PATH);
db.pragma('journal_mode = WAL');
db.exec(`
CREATE TABLE IF NOT EXISTS users (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 email TEXT NOT NULL UNIQUE,
 password_hash TEXT NOT NULL,
 role TEXT NOT NULL DEFAULT 'user',
 identity_status TEXT NOT NULL DEFAULT 'not_submitted',
 created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS properties (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 user_id INTEGER NOT NULL REFERENCES users(id),
 title TEXT NOT NULL,
 description TEXT NOT NULL,
 property_type TEXT NOT NULL,
 listing_type TEXT NOT NULL,
 country TEXT NOT NULL,
 city TEXT NOT NULL,
 price REAL NOT NULL,
 currency TEXT NOT NULL DEFAULT 'EUR',
 bedrooms INTEGER NOT NULL DEFAULT 0,
 area REAL NOT NULL DEFAULT 0,
 image_path TEXT,
 verification_status TEXT NOT NULL DEFAULT 'pending',
 created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
`);
```

```
// Demo listings are clearly marked as sample data and are not
represented as verified.
const count = db.prepare('SELECT COUNT(*) AS n FROM
properties').get().n;
if (count === 0) {
  const demoUser = db.prepare('INSERT INTO users
(name,email,password_hash,role) VALUES (?,?,?,?)')
    .run('Universal Estate Demo', 'demo@universalestate.test',
bcrypt.hashSync('Demo-Only-Password-123!', 10), 'user');
  const add = db.prepare(`INSERT INTO properties

(user_id,title,description,property_type,listing_type,country,c
ity,price,currency,bedrooms,area,image_path,verification_status
)
    VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)`);
  add.run(demoUser.lastInsertRowid,'Maison contemporaine avec
jardin','Exemple de fiche de dÃ©monstration. Contactez le
vendeur pour obtenir les documents
officiels.','Maison','Vente','France','Bordeaux',389000,'EUR',4
,142,'https://images.unsplash.com/photo-1600585154340-
be6161a56a0c?auto=format&fit=crop&w=1200&q=80','pending');
  add.run(demoUser.lastInsertRowid,'Appartement lumineux prÃ¨s
du centre','Exemple de fiche de dÃ©monstration. Cette annonce
est fictive et non
vÃ©rifiÃ©e.','Appartement','Vente','Espagne','Valence',245000,'
EUR',2,86,'https://images.unsplash.com/photo-1600607687939-
ce8a6c25118c?auto=format&fit=crop&w=1200&q=80','pending');
  add.run(demoUser.lastInsertRowid,'Villa avec
piscine','Exemple de fiche de dÃ©monstration. VÃ©rifiez
toujours lâidentitÃ© du vendeur et les titres de
propriÃ©tÃ©.','Villa','Vente','SÃ©nÃ©gal','Dakar',175000000,'XO
F',5,210,'https://images.unsplash.com/
photo-1613490493576-7fde63acd811?
auto=format&fit=crop&w=1200&q=80','pending');
}

app.disable('x-powered-by');
app.use(helmet({ contentSecurityPolicy: false }));
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true, limit: '1mb' }));
app.use(session({
  name: 'ue.sid',
  secret: process.env.SESSION_SECRET || 'dev-only-change-this-
secret-before-deploying',
```

```
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, sameSite: 'lax', secure:
process.env.NODE_ENV === 'production', maxAge: 1000 * 60 * 60 *
8 }
}));
app.use('/uploads', express.static(UPLOAD_DIR));
app.use(express.static(path.join(__dirname, 'public')));

const upload = multer({
  dest: UPLOAD_DIR,
  limits: { fileSize: 5 * 1024 * 1024, files: 1 },
  fileFilter: (_req, file, cb) => {
    const allowed = ['image/jpeg', 'image/png', 'image/webp'];
    cb(allowed.includes(file.mimetype) ? null : new
Error('Format acceptÃ© : JPG, PNG ou WebP.'),
allowed.includes(file.mimetype));
  }
});
const safeUser = u => u && ({ id: u.id, name: u.name, email:
u.email, role: u.role, identityStatus: u.identity_status });
function requireAuth(req, res, next) {
  if (!req.session.userId) return res.status(401).json({ error:
'Connectez-vous pour continuer.' });
  next();
}
function requireAdmin(req, res, next) {
  const user = req.session.userId ? db.prepare('SELECT * FROM
users WHERE id=?').get(req.session.userId) : null;
  if (!user || user.role !== 'admin') return
res.status(403).json({ error: 'AccÃ¨s rÃ©servÃ© Ã  la
modÃ©ration.' });
  req.admin = user; next();
}

app.get('/api/me', (req, res) => {
  const u = req.session.userId ? db.prepare('SELECT * FROM
users WHERE id=?').get(req.session.userId) : null;
  res.json({ user: safeUser(u) });
});
app.post('/api/register', async (req, res) => {
  const name = String(req.body.name || '').trim();
  const email = String(req.body.email ||
'').trim().toLowerCase();
  const password = String(req.body.password || '');
```

```
  if (name.length < 2 || name.length > 80) return
res.status(400).json({ error: 'Le nom doit contenir entre 2 et
80 caractÃ¨res.' });
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) || email.length
> 190) return res.status(400).json({ error: 'Adresse e-mail
invalide.' });
  if (password.length < 12) return
res.status(400).json({ error: 'Choisissez un mot de passe de 12
caractÃ¨res minimum.' });
  try {
    const hash = await bcrypt.hash(password, 12);
    const result = db.prepare('INSERT INTO users
(name,email,password_hash) VALUES (?,?,?)').run(name, email,
hash);
    req.session.userId = result.lastInsertRowid;
    res.status(201).json({ user: safeUser(db.prepare('SELECT *
FROM users WHERE id=?').get(result.lastInsertRowid)) });
  } catch (e) {
    if (String(e.message).includes('UNIQUE')) return
res.status(409).json({ error: 'Cette adresse e-mail est dÃ©jÃ
utilisÃ©e.' });
    res.status(500).json({ error: 'Inscription impossible pour
le moment.' });
  }
});
app.post('/api/login', async (req, res) => {
  const email = String(req.body.email ||
'').trim().toLowerCase();
  const password = String(req.body.password || '');
  const user = db.prepare('SELECT * FROM users WHERE
email=?').get(email);
  if (!user || !(await bcrypt.compare(password,
user.password_hash))) return res.status(401).json({ error: 'E-
mail ou mot de passe incorrect.' });
  req.session.userId = user.id;
  res.json({ user: safeUser(user) });
});
app.post('/api/logout', (req, res) => req.session.destroy(() =>
res.json({ ok: true })));

app.get('/api/properties', (req, res) => {
  const where = ["p.verification_status IN
('verified','pending')"];
  const params = [];
  const q = String(req.query.q || '').trim();
```

```
  const country = String(req.query.country || '').trim();
  const type = String(req.query.type || '').trim();
  const min = Number(req.query.min || 0);
  const max = Number(req.query.max || 0);
  if (q) { where.push('(p.city LIKE ? OR p.country LIKE ? OR
p.title LIKE ?)'); params.push(`%${q}%`,`%${q}%`,`%${q}%`); }
  if (country) { where.push('p.country = ?');
params.push(country); }
  if (type) { where.push('p.property_type = ?');
params.push(type); }
  if (min > 0) { where.push('p.price >= ?');
params.push(min); }
  if (max > 0) { where.push('p.price <= ?');
params.push(max); }
  const rows = db.prepare(`SELECT
p.id,p.title,p.description,p.property_type,p.listing_type,p.cou
ntry,p.city,p.price,p.currency,p.bedrooms,p.area,p.image_path,p
.verification_status,p.created_at,u.name AS seller_name FROM
properties p JOIN users u ON u.id=p.user_id WHERE $
{where.join(' AND ')} ORDER BY p.created_at DESC LIMIT
100`).all(...params);
  res.json({ properties: rows });
});
app.get('/api/my-properties', requireAuth, (req, res) => {
  res.json({ properties: db.prepare('SELECT * FROM properties
WHERE user_id=? ORDER BY created_at
DESC').all(req.session.userId) });
});
app.post('/api/properties', requireAuth, (req, res) => {
  const b = req.body;
  const title = String(b.title || '').trim();
  const description = String(b.description || '').trim();
  const propertyType = String(b.propertyType || 'Maison');
  const listingType = String(b.listingType || 'Vente');
  const country = String(b.country || '').trim();
  const city = String(b.city || '').trim();
  const price = Number(b.price);
  const currency = String(b.currency || 'EUR');
  const bedrooms = Math.max(0, Math.min(100, Number(b.bedrooms
|| 0)));
  const area = Math.max(0, Math.min(1000000, Number(b.area ||
0)));
  if (title.length < 5 || title.length > 120 ||
description.length < 20 || description.length > 5000 || !
country || country.length > 80 || !city || city.length > 100
```

```
|| !Number.isFinite(price) || price <= 0) {
    return res.status(400).json({ error: 'VÃ©rifiez le titre,
la description, le pays, la ville et le prix.' });
  }
  if (!
['Maison','Appartement','Terrain','Immeuble','Villa','Autre'].i
ncludes(propertyType)) return res.status(400).json({ error:
'Type de bien invalide.' });
  if (!['Vente','Location'].includes(listingType)) return
res.status(400).json({ error: 'Type dâannonce invalide.' });
  if (!
['EUR','USD','XOF','GBP','CAD','AED'].includes(currency))
return res.status(400).json({ error: 'Devise non prise en
charge.' });
  const result = db.prepare(`INSERT INTO properties
(user_id,title,description,property_type,listing_type,country,c
ity,price,currency,bedrooms,area,verification_status) VALUES
(?,?,?,?,?,?,?,?,?,?,?,'pending')`)
    .run(req.session.userId,title,description,propertyType,list
ingType,country,city,price,currency,bedrooms,area);
  res.status(201).json({ id: result.lastInsertRowid, message:
'Annonce reÃ§ue : elle sera publiÃ©e aprÃ¨s modÃ©ration.' });
});
app.post('/api/identity-review-request', requireAuth, (req,
res) => {
  // Do not collect identity documents in this starter. A
compliant verification provider/workflow must be configured
first.
  db.prepare("UPDATE users SET
identity_status='review_requested' WHERE
id=?").run(req.session.userId);
  res.json({ message: 'Demande enregistrÃ©e. Aucun document
dâidentitÃ© nâa Ã©tÃ© collectÃ© par cette version de
dÃ©monstration.' });
});
app.get('/api/admin/pending', requireAdmin, (_req, res) => {
  res.json({ properties: db.prepare(`SELECT p.*,u.name AS
seller_name,u.email AS seller_email FROM properties p JOIN
users u ON u.id=p.user_id WHERE p.verification_status='pending'
ORDER BY p.created_at ASC`).all() });
});
app.post('/api/admin/properties/:id/decision', requireAdmin,
(req, res) => {
  const status = String(req.body.status || '');
  if (!['verified','rejected'].includes(status)) return
```

```
res.status(400).json({ error: 'DÃ©cision invalide.' });
  const result = db.prepare("UPDATE properties SET
verification_status=? WHERE id=? AND
verification_status='pending'").run(status,
Number(req.params.id));
  if (!result.changes) return res.status(404).json({ error:
'Annonce introuvable ou dÃ©jÃ  traitÃ©e.' });
  res.json({ ok: true });
});

// Optional first-run administrator. Configure both variables
before launch; never use defaults in production.
if (process.env.ADMIN_EMAIL && process.env.ADMIN_PASSWORD) {
  const email = process.env.ADMIN_EMAIL.toLowerCase();
  if (!db.prepare('SELECT id FROM users WHERE
email=?').get(email)) {
    const hash = bcrypt.hashSync(process.env.ADMIN_PASSWORD,
12);
    db.prepare("INSERT INTO users
(name,email,password_hash,role) VALUES
(?,?,?,'admin')").run('Administrateur', email, hash);
  }
}
app.use((err, _req, res, _next) => {
  console.error(err.message);
  res.status(400).json({ error: err.message || 'Une erreur est
survenue.' });
});
app.listen(PORT, () => console.log(`Universal Estate disponible
sur http://localhost:${PORT}`));
```